Assignment 3: Build Unit Test Automation Solution

Name: Ali Asif | Student ID: 22F-3710 | Course: Software Quality Engineering
Submission: solo (individual) | Repository: https://github.com/Ali199012/unit-test-automation

1. Project

The project under test is **cachetools**, an open-source Python library that provides caches (LRUCache, TTLCache and others) and caching decorators. Source: https://github.com/tkem/cachetools

2. Setup (Task 1)

I installed Python and Git, cloned the project, created a virtual environment, installed the project with `pip install -e .`, and installed pytest, pytest-xdist, pytest-order, pandas and openpyxl. The project's own tests ran successfully. The setup video is `docs/Setup.mp4`.

Problems and fixes: the first video was 246 MB, over GitHub's 100 MB file limit, so I compressed it. The first push failed with an HTTP 408 timeout, and I fixed it by pushing in smaller parts. One test run failed with "No module named cachetools" because the virtual environment was not activated.

3. Modules and techniques (Task 2)

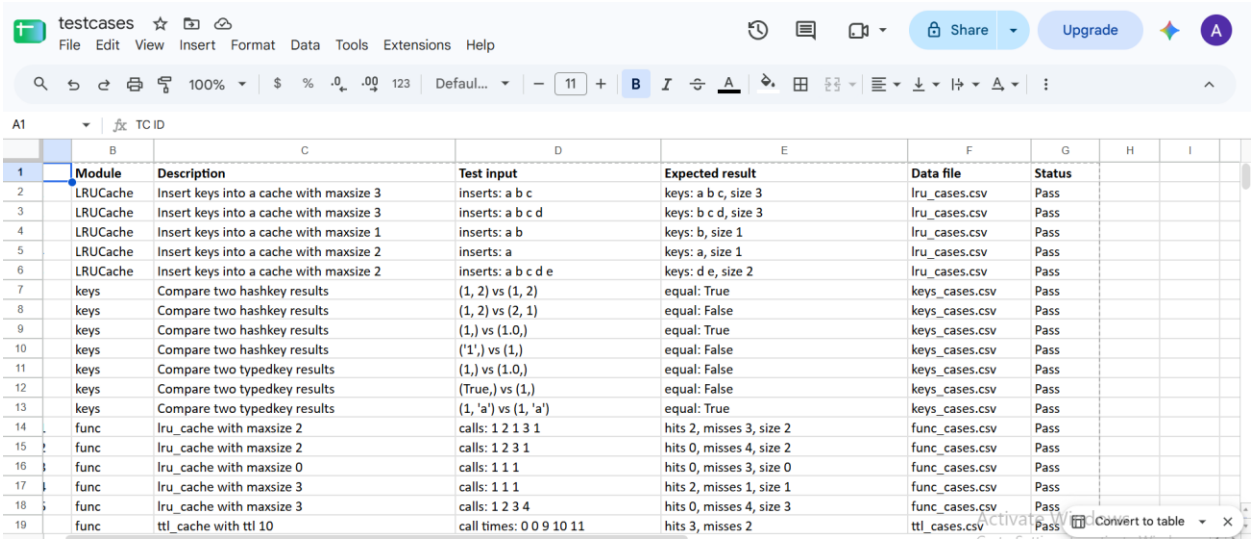
I chose three modules and wrote ECP and BVA notes for each before writing the tests (`docs/notes_lru.md`, `docs/notes_keys.md`, `docs/notes_func.md`). Tests use pytest with `pytest.mark.parametrize`.

Module	What was tested	Test file	Test cases
LRUCache (__init__.py)	__setitem__, __getitem__, popitem, class level	tests/test_lru_mine.py	34
keys.py	hashkey, typedkey, module level	tests/test_keys_mine.py	25
func.py	lru_cache, ttl_cache, module level	tests/test_func_mine.py	25

Examples of what I tested: ECP classes such as valid keys, missing keys and unhashable keys; BVA values such as maxsize 0, 1, 2, a full cache plus one item, and a TTL of exactly 10 (an entry expires at time 10, not at 11). Something I noticed: `hashkey(1)` equals `hashkey(1.0)`, while `typedkey` keeps them apart.

4. Data-driven tests and Excel (Task 3)

Four tests in `tests/test_csv_mine.py` read their data from four CSV files in `tests/data/` (lru_cases, keys_cases, func_cases, ttl_cases), 21 cases in total. The test cases are listed in `docs/testcases.xlsx`.



testcases

File Edit View Insert Format Data Tools Extensions Help

Share Upgrade

A1 TC ID

	Module	Description	Test input	Expected result	Data file	Status
2	LRUCache	Insert keys into a cache with maxsize 3	inserts: a b c	keys: a b c, size 3	lru_cases.csv	Pass
3	LRUCache	Insert keys into a cache with maxsize 3	inserts: a b c d	keys: b c d, size 3	lru_cases.csv	Pass
4	LRUCache	Insert keys into a cache with maxsize 1	inserts: a b	keys: b, size 1	lru_cases.csv	Pass
5	LRUCache	Insert keys into a cache with maxsize 2	inserts: a	keys: a, size 1	lru_cases.csv	Pass
6	LRUCache	Insert keys into a cache with maxsize 2	inserts: a b c d e	keys: d e, size 2	lru_cases.csv	Pass
7	keys	Compare two hashkey results	(1, 2) vs (1, 2)	equal: True	keys_cases.csv	Pass
8	keys	Compare two hashkey results	(1, 2) vs (2, 1)	equal: False	keys_cases.csv	Pass
9	keys	Compare two hashkey results	(1,) vs (1.0,)	equal: True	keys_cases.csv	Pass
10	keys	Compare two hashkey results	('1',) vs (1,)	equal: False	keys_cases.csv	Pass
11	keys	Compare two typedkey results	(1,) vs (1.0,)	equal: False	keys_cases.csv	Pass
12	keys	Compare two typedkey results	(True,) vs (1,)	equal: False	keys_cases.csv	Pass
13	keys	Compare two typedkey results	(1, 'a') vs (1, 'a')	equal: True	keys_cases.csv	Pass
14	func	lru_cache with maxsize 2	calls: 1 2 1 3 1	hits 2, misses 3, size 2	func_cases.csv	Pass
15	func	lru_cache with maxsize 2	calls: 1 2 3 1	hits 0, misses 4, size 2	func_cases.csv	Pass
16	func	lru_cache with maxsize 0	calls: 1 1 1	hits 0, misses 3, size 0	func_cases.csv	Pass
17	func	lru_cache with maxsize 3	calls: 1 1 1	hits 2, misses 1, size 1	func_cases.csv	Pass
18	func	lru_cache with maxsize 3	calls: 1 2 3 4	hits 0, misses 4, size 3	func_cases.csv	Pass
19	func	ttl_cache with ttl 10	call times: 0 0 9 10 11	hits 3, misses 2	ttl_cases.csv	Pass

Convert to table

5. Configuration (Task 4)

- **Groups:** six markers in `pytest.ini` (lru, keys, func, datadriven, boundary, negative), assigned in `tests/conftest.py`. Example: `pytest -m boundary`.
- **Execution order:** set in `conftest.py`: LRU, then keys, then func, then CSV tests.
- **Parallel runs:** `pytest -n auto` (pytest-xdist).
- **Parameterization:** used across all test files, and data comes from CSV in Task 3.
- All commands are in `TESTING.md`.

```
(.venv) C:\Users\X\Desktop\cachetools>pytest -m keys
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\X\Desktop\cachetools
configfile: pytest.ini
testpaths: tests
plugins: order-1.5.0, xdist-3.8.0
collected 111 items / 86 deselected / 25 selected

tests\test_keys_mine.py .........................                                                    [100%]

========================================== 25 passed, 86 deselected in 0.11s ==========================================

(.venv) C:\Users\X\Desktop\cachetools>
```

```
(.venv) C:\Users\X\Desktop\cachetools>pytest -m boundary
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\X\Desktop\cachetools
configfile: pytest.ini
testpaths: tests
plugins: order-1.5.0, xdist-3.8.0
collected 111 items / 79 deselected / 32 selected

tests\test_lru_mine.py ..............                                                                [ 43%]
tests\test_keys_mine.py ..                                                                           [ 50%]
tests\test_func_mine.py ............                                                                 [ 87%]
tests\test_csv_mine.py ....                                                                          [100%]

========================================= 32 passed, 79 deselected in 0.14s =========================================

(.venv) C:\Users\X\Desktop\cachetools>
```

```
========================================= 32 passed, 79 deselected in 0.14s =========================================

(.venv) C:\Users\X\Desktop\cachetools>pytest -n auto
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\X\Desktop\cachetools
configfile: pytest.ini
testpaths: tests
plugins: order-1.5.0, xdist-3.8.0
4 workers [111 items]
...........................................................................................................    [100%]
============================================= 111 passed in 2.43s =============================================

(.venv) C:\Users\X\Desktop\cachetools>
```

6. Results

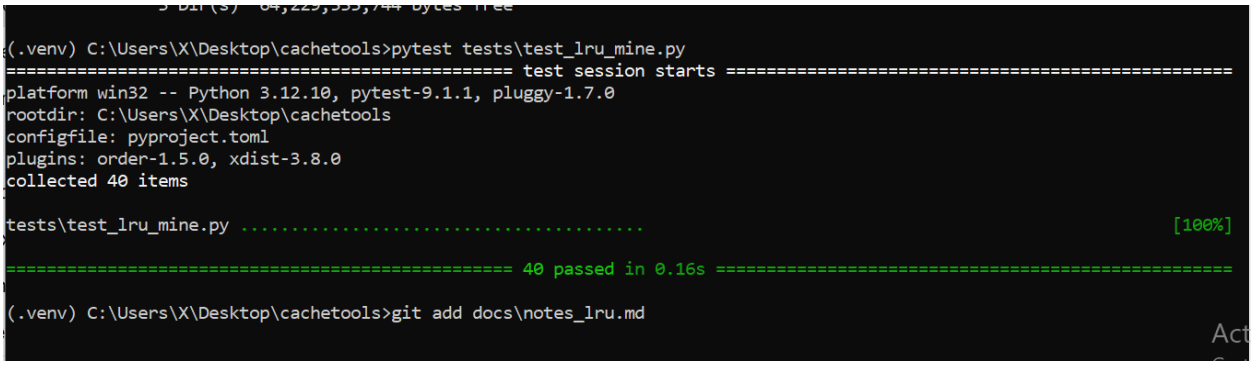
105 test cases ran and all passed (34 LRUCache, 25 keys, 25 func, 21 data-driven). The red marks on GitHub come from the original project's CI configuration, which fails on my repository; the tests themselves pass locally, as the screenshots show.

```
(.venv) C:\Users\X\Desktop\cachetools>pytest
============================================= test session starts =============================================
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Users\X\Desktop\cachetools
configfile: pytest.ini
testpaths: tests
plugins: order-1.5.0, xdist-3.8.0
collected 105 items

tests\test_lru_mine.py ..................................                                            [ 32%]
tests\test_keys_mine.py ........................                                                     [ 56%]
tests\test_func_mine.py ........................                                                     [ 80%]
tests\test_csv_mine.py .....................                                                         [100%]

============================================= 105 passed in 0.28s =============================================

(.venv) C:\Users\X\Desktop\cachetools>
```



7. Contribution and AI use

This is a solo submission, so all the work is mine. The repository also contains the original cachetools commit history; my own commits start on Oct 25, 2026. I used an AI assistant (Claude) for guidance. I ran every test myself and I can explain each part of the code.